

Top-Down Design Hierarchy

Scalable shared-memory stereo block-matching accelerator:
responsibilities, interfaces, implementation contracts,
verification, and integration gates

Target platform	Terasic DE2-115 · Cyclone IV EP4CE115
Compute system	Nios V CPU + parameterized SAD accelerator + shared SDRAM
Abstraction	System mission → partition → subsystems → datapath → RTL → build gates
Team	Three members with explicit integration boundaries

Contents

Top-Down Design Hierarchy

Scalable shared-memory stereo block-matching accelerator

1. How to use the hierarchy

Level 0 — Complete-system mission

2. System contract

2.1 Inputs and outputs

2.2 Definition of done

Level 1 — System partition

3. Major components and their contributions

3.1 A. Nios V software subsystem

3.2 B. FPGA platform subsystem

3.3 C. Domain accelerator IP

3.4 D. Shared SDRAM data subsystem

3.5 Level-1 interconnect contracts

Level 2 — Accelerator subsystem hierarchy

4. Subsystem data and control flow

4.1 Block 1 — Control plane

4.2 Block 2 — Frame/disparity-group scheduler

4.3 Block 3 — Address generator

4.4 Block 4 — Read frontend

4.5 Block 5 — Reuse frontend

4.6 Block 6 — SAD compute core

4.7 Block 7 — Write backend

4.8 Block 8 — Instrumentation and observability

4.9 Internal stream contract

Level 3 — Parameterized SAD datapath

5. Per-item transformation

5.1 Datapath children

5.2 Compile-time parameters

5.3 Scalability mechanism

Level 4 — Proposed RTL module hierarchy

6. Module tree

6.1 Top-level external ports

6.2 Proposed module contracts

Level 5 — Implementation and integration order

7. Vertical-slice build gates

7.1 Why this order is safer

7.2 Team partition with explicit boundaries

Platform/integration owner

Datapath/RTL owner

Software/evaluation owner

Shared responsibilities

8. Clean project outline for proposal, report, and viva

9. Immediate next action

Top-Down Design Hierarchy

Scalable shared-memory stereo block-matching accelerator

This document is the **architecture spine** of the project. It separates the design into levels so that requirements, system integration, RTL implementation, and testing are not discussed as if they were the same thing.

The rule is:

At each level, define the block's responsibility, interfaces, implementation, and verification before opening the block and discussing its children.

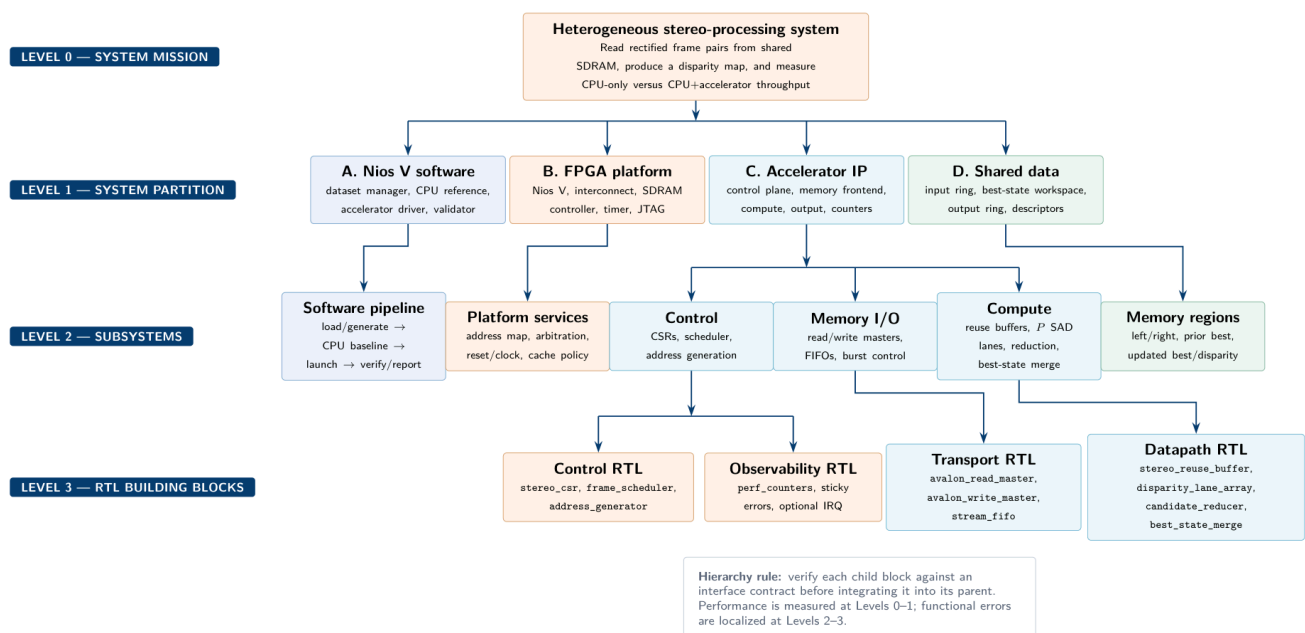


Figure 1 — The project decomposed from its measurable system mission to independently testable RTL building blocks.

1. How to use the hierarchy

Level	Question answered	Main deliverable
0 — Mission	What must the complete system demonstrate?	End-to-end functional and performance contract
1 — System partition	Which major CPU, platform, accelerator, and memory blocks cooperate?	Platform Designer system and software/hardware boundary
2 — Accelerator subsystems	Which internal services make the accelerator work?	Control, transport, compute, output, and instrumentation contracts
3 — Datapath	How is one stream item transformed cycle by cycle?	Parameterized SAD pipeline and fixed-point widths
4 — RTL modules	What independently testable modules should be coded?	Proposed SystemVerilog module tree and unit tests
5 — Build gates	In what order is the hierarchy integrated?	Incremental milestones with objective pass/fail criteria

Do not jump from Level 0 directly to RTL. For example, “accelerate stereo matching” is not enough information to implement an Avalon master, while “write `absdiff.sv`” says nothing about shared-memory compliance or end-to-end speedup.

Level 0 — Complete-system mission

2. System contract

The complete system shall:

1. Hold rectified left/right grayscale frame pairs in external SDRAM.
2. Allow the Nios V CPU and the accelerator to access the **same SDRAM address space** through the Platform Designer interconnect.
3. Produce an integer disparity map using the agreed SAD algorithm, valid-region policy, tie rule, and output format.
4. Run a mathematically equivalent CPU-only implementation over the same SDRAM-resident inputs and outputs.

5. Measure continuous-batch throughput using a cycle counter, including accelerator setup, bus arbitration, memory traffic, compute, output commit, and completion synchronization.
6. Demonstrate scalability by compiling multiple values of `P_LANES` and plotting resource use against throughput/speedup.

2.1 Inputs and outputs

Item	Representation	Owner
Left frame	8-bit rectified grayscale, row-major, explicit stride	Shared SDRAM
Right frame	8-bit rectified grayscale, row-major, explicit stride	Shared SDRAM
Configuration	Base addresses, dimensions, strides, window, disparity count	CPU-written CSRs
Output disparity	Unsigned disparity index; width derived from <code>D_MAX</code>	Shared SDRAM
Optional best cost	Unsigned SAD cost; width derived from window and pixel width	Shared SDRAM workspace
Performance result	Total/active/stall cycles, accepted transfers, output pixels	Accelerator counters + CPU timer

2.2 Definition of done

The system is complete only when all of the following are true:

- RTL output matches the integer software golden model bit-for-bit on directed and dataset tests.
 - CPU and accelerator access the same SDRAM buffers; there is no CPU-managed copy into a private accelerator frame store.
 - A batch of frame pairs runs continuously without reprogramming the FPGA.
 - CPU-only and accelerated timings use the same timing boundary.
 - At least three `P_LANES` builds provide `fMAX`, logic, memory bits, DSP count, throughput, and speedup.
-

Level 1 — System partition

3. Major components and their contributions

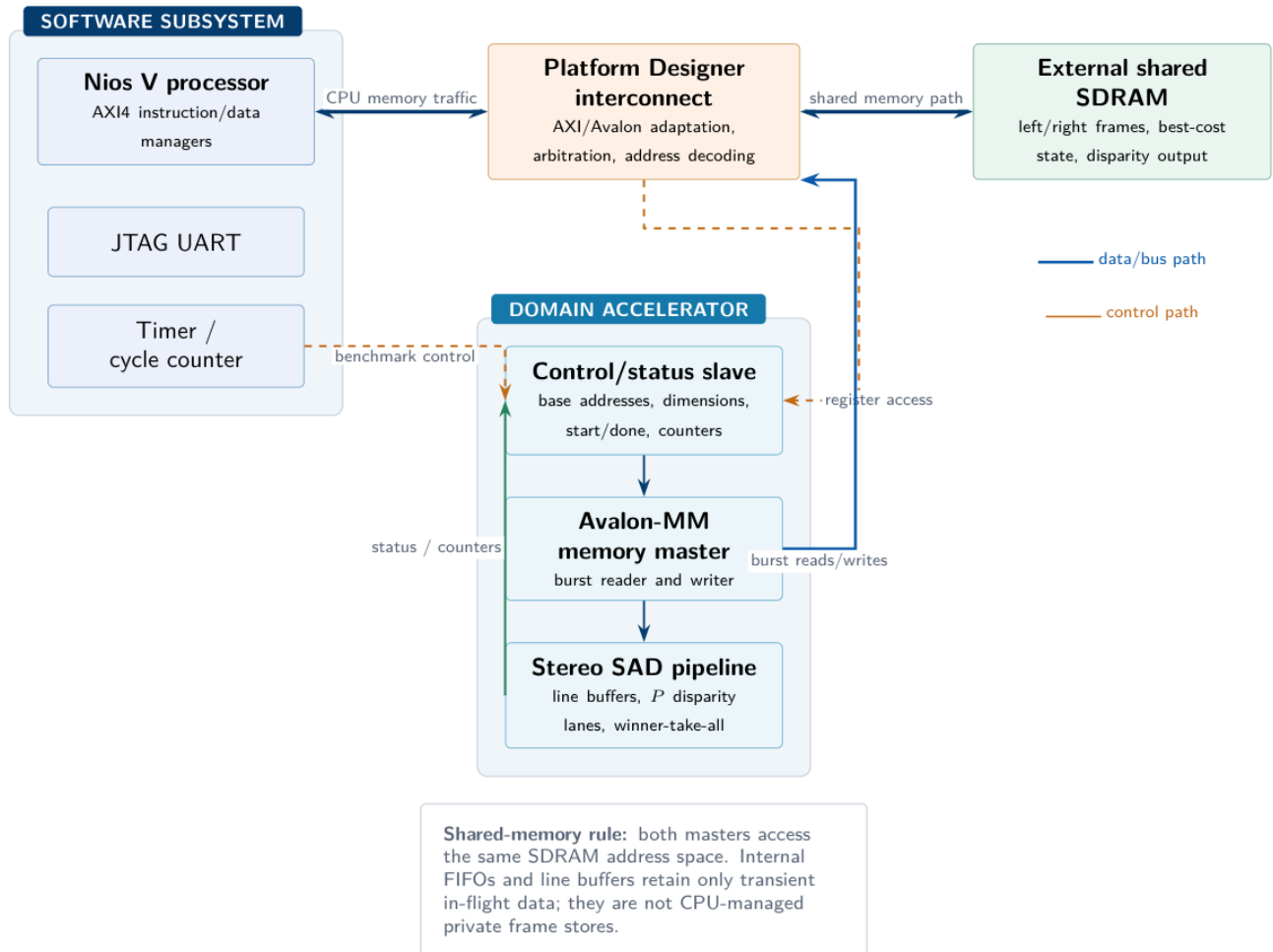


Figure 2 — Nios V, Platform Designer, accelerator, and external SDRAM at the hardware/software boundary.

3.1 A. Nios V software subsystem

Contribution: supplies general-purpose control, the CPU baseline, datasets, verification, and presentation. It proves that the design is a heterogeneous CPU+accelerator system rather than a standalone FPGA datapath.

Implement as:

- Dataset loader or synthetic-frame generator.
- Integer CPU SAD reference using the frozen arithmetic contract.
- Accelerator driver for configuration, launch, completion, and cache maintenance.
- Output comparator and error reporter.
- Benchmark harness that repeats frame batches and prints cycle/throughput metrics.
- Optional VGA/display controller code outside the measured region.

Verify by: running the CPU reference against precomputed tiny vectors before using it as the RTL oracle; reading/writing all accelerator CSRs; testing changing input patterns to expose stale-cache errors.

3.2 B. FPGA platform subsystem

Contribution: provides processor execution, address decoding, arbitration, clock/reset, memory control, and observability services used by both CPU and accelerator.

Implement in Platform Designer as:

- Nios V processor and its instruction/data paths.
- On-chip boot/program memory if required.
- External SDRAM controller.
- AXI/Avalon adaptation generated by Platform Designer.
- Custom accelerator control slave and memory master connections.
- JTAG UART and timer/cycle-counter path.
- Optional IRQ and VGA path.

Verify by: preserving the current Hello World baseline; then proving SDRAM from the CPU; then integrating a scratch-register slave; then integrating a pass-through custom memory master.

3.3 C. Domain accelerator IP

Contribution: removes the repeated disparity/window computation from the CPU and exploits deterministic streaming reuse and `P_LANES` parallelism.

Implement as: one custom component with a small control/status slave, an autonomous SDRAM memory master, an optional IRQ, and the internal Level-2 subsystems described later.

Verify by: testing the control plane independently, transport with a pass-through transform, one SAD lane, full disparity search, and finally multi-lane scaling.

3.4 D. Shared SDRAM data subsystem

Contribution: is the common producer-consumer storage and the key systems constraint. It also makes CPU-only and accelerated paths comparable.

Implement as fixed software-managed regions:

- Descriptor/ring metadata.
- Left-frame buffers A/B.
- Right-frame buffers A/B.
- Prior/updated best-cost state if the baseline uses disparity-group passes.
- Output disparity buffers A/B.
- Optional CPU reference output.

Internal FIFOs and line/history buffers are allowed only as transient reuse and elasticity structures. They are not software-visible frame stores.

3.5 Level-1 interconnect contracts

Connection	Protocol/direction	Purpose
Nios V → Platform interconnect	Generated CPU manager interface	Instruction/data and SDRAM traffic
CPU → accelerator control plane	Memory-mapped slave access	Configure, start, poll, read counters
Accelerator → interconnect → SDRAM	Avalon-MM memory master	Autonomous burst reads/writes
Accelerator → CPU	Optional interrupt	Completion/error notification
CPU ↔ SDRAM	Shared address space	Dataset, CPU baseline, verification
Accelerator ↔ SDRAM	Same shared address space	Inputs, state, and disparity output

The CPU path and accelerator path may use different protocol adapters, but they must converge on the same SDRAM controller and address map.

Level 2 — Accelerator subsystem hierarchy

4. Subsystem data and control flow

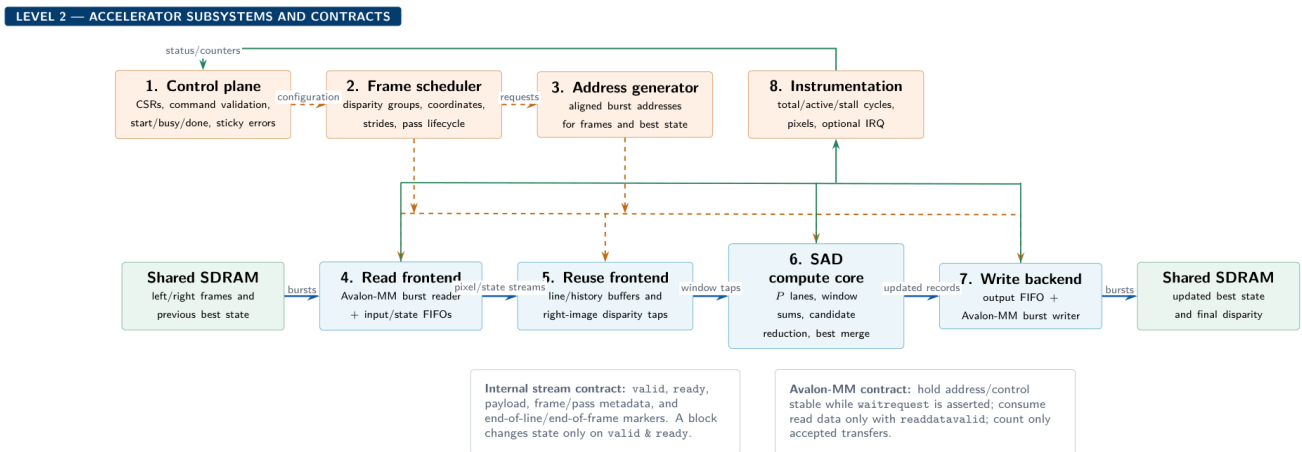


Figure 3 — Accelerator decomposition. Blue is the high-volume data path, orange is command/control, and green is status/instrumentation.

4.1 Block 1 — Control plane

Responsibility: provide the only CPU-visible accelerator interface.

Inputs: Avalon-MM CSR reads/writes, reset, scheduler status, error events, performance counters.

Outputs: validated configuration snapshot, start pulse, clear/reset requests, interrupt enable.

Implementation:

- Identification/version and capability registers.
- Base address, width, height, stride, `D_MAX`, window, and frame-count registers.
- Start/busy/done/error semantics.
- Configuration validation for alignment, range, dimensions, and unsupported combinations.
- Shadow configuration latched only when a command is accepted, so CPU writes cannot alter an active frame.

Unit verification: register read/write tests, illegal-command tests, start-while-busy behavior, sticky-error clearing, and stable configuration during operation.

4.2 Block 2 — Frame/disparity-group scheduler

Responsibility: convert one CPU command into deterministic frame, pass, row, and completion sequencing.

Inputs: configuration snapshot, downstream readiness, transport completion, error events.

Outputs: pass index, disparity-group base, row/column validity, read/write requests, done/error.

Recommended FSM:

IDLE → VALIDATE → INIT_FRAME → INIT_GROUP → STREAM → DRAIN → NEXT_GROUP/NEXT_FRAME → DONE

Any fatal bus/FIFO/configuration event transitions to `ERROR` until software clears it.

Unit verification: tiny dimensions, first/last group, non-multiple `D_MAX/P_LANES`, row boundaries, frame batches, backpressure, and error abort.

4.3 Block 3 — Address generator

Responsibility: translate frame/pass coordinates into aligned SDRAM transactions without embedding bus-handshake logic in the scheduler.

Inputs: base addresses, strides, dimensions, group/pass index, transfer requests.

Outputs: address, burst length, byte enables, and transfer metadata.

Implementation: maintain separate logical generators for left pixels, right pixels, prior best state, and updated output state; split bursts at row, buffer, and controller boundaries.

Unit verification: first/last pixel, row-stride padding, burst boundary split, alignment rejection, and no out-of-region access.

4.4 Block 4 — Read frontend

Responsibility: turn shared-SDRAM transactions into elastic internal streams.

Inputs: read commands, `waitrequest`, `readdata`, `readdatavalid`, downstream `ready`.

Outputs: left/right pixel stream, prior-best stream, FIFO level/stall/error events.

Implementation: Avalon-MM read master, outstanding-read accounting, response tagging if streams are interleaved, and input/state FIFOs.

Critical protocol rule: hold command/address stable while stalled and enqueue data only on `readdatavalid`.

Unit verification: randomized wait states and response latency, FIFO almost-full throttling, maximum bursts, row transitions, and data-order scoreboard.

4.5 Block 5 — Reuse frontend

Responsibility: convert raster-order pixels into the spatial and disparity taps needed by the SAD lanes while retaining only transient data.

Inputs: accepted pixel stream and scheduler metadata.

Outputs: aligned left pixel, `P_LANES` right-image disparity taps, window-history values, valid-region metadata.

Implementation:

- Left/right row storage needed by the window.
- Right-image horizontal history sufficient for active disparities.
- Shift-register or RAM-based horizontal window state.
- Explicit warm-up/flush validity at left, right, top, and bottom borders.

Unit verification: impulse images, row changes, multiple image widths, each disparity tap, border suppression, and arbitrary backpressure.

4.6 Block 6 — SAD compute core

Responsibility: produce one group winner for each valid output coordinate and merge it with prior best state.

Children: `disparity_lane_array`, `candidate_reducer`, and `best_state_merge`.

Inputs: aligned taps, disparity-group base, prior best record, valid/ready.

Outputs: updated `(best_cost, best_disparity, x, y, valid)` records.

Implementation: detailed at Level 3. Arithmetic is unsigned and fixed-width; equal costs use one frozen tie rule, preferably the lower disparity.

Unit verification: lane-level arithmetic, reducer permutations/ties, prior-best merge, last partial group, and stalls at every pipeline stage.

4.7 Block 7 — Write backend

Responsibility: commit updated state and final disparity records to the specified shared-SDRAM region.

Inputs: output-record stream, addresses, `waitrequest`.

Outputs: Avalon-MM writes, FIFO/stall/error events, completion only after the last accepted write.

Implementation: output FIFO, packing/byte-enable logic, burst coalescing, and accepted-write accounting.

Unit verification: randomized stalls, partial final burst, stride padding, output packing, and proof that `done` never precedes the final accepted write.

4.8 Block 8 — Instrumentation and observability

Responsibility: explain performance and failures rather than providing only a final elapsed time.

Counters: total cycles, datapath-active cycles, read-stall cycles, write-stall cycles, accepted read/write beats, valid output pixels, frames, and optionally FIFO high-water marks.

Implementation: saturating or wide counters with explicit reset/snapshot behavior; sticky error bits; optional completion interrupt.

Unit verification: one-event-per-cycle tests, simultaneous events, reset/snapshot, saturation, and consistency checks such as output-pixel count.

4.9 Internal stream contract

Every internal pipeline boundary should use one common convention:

Signal	Meaning
<code>valid</code>	Producer currently presents a meaningful item
<code>ready</code>	Consumer can accept the item this cycle
<code>payload</code>	Pixels, costs, disparity, or output state
<code>x, y</code>	Coordinate metadata where required
<code>sof, eol, eof</code>	Frame/row boundary metadata where useful
<code>pass_id / group_base</code>	Disparity-group metadata

A transfer occurs only on `valid && ready`. A stateful block must either stall all corresponding data/metadata state together or place an elasticity FIFO at the boundary.

Level 3 — Parameterized SAD datapath

5. Per-item transformation

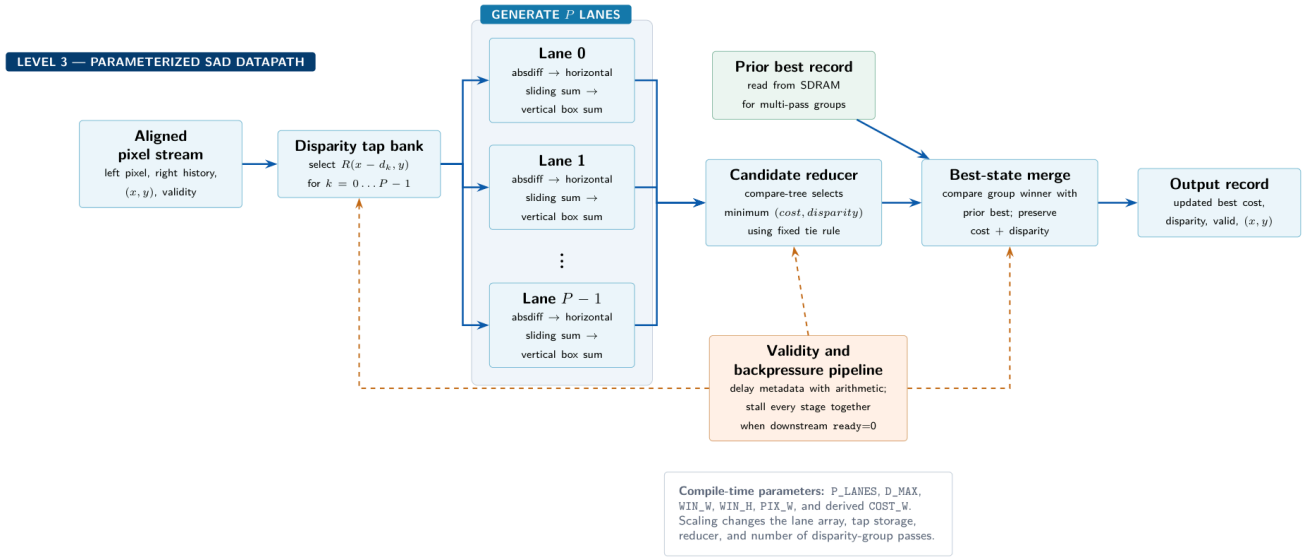


Figure 4 — One group of P_LANES disparities is evaluated in parallel and merged with the prior best record.

For lane k in a group with base disparity g :

$$d_k = g + k$$

$$A_{d_k}(x, y) = |L(x, y) - R(x - d_k, y)|$$

$$C(x, y, d_k) = \sum_{i=-r_y}^{r_y} \sum_{j=-r_x}^{r_x} A_{d_k}(x + j, y + i)$$

The lane emits (cost, disparity, valid). A comparator tree chooses the group winner, then `best_state_merge` compares that winner against the prior best record from SDRAM. The updated record returns to SDRAM for the next group or becomes the final disparity output after the last group.

5.1 Datapath children

Child	Contribution	Implementation idea	Directed test
Disparity tap bank	Supplies $R(x-d_k, y)$ for each lane	Right-history shift/RAM taps indexed by group base + lane	Ramp row; check every tap
absdiff	Per-pixel matching cost	Unsigned subtract + magnitude/mux	0/255, equal, swapped operands
Horizontal sliding sum	Reuses adjacent window columns	Add newest, subtract oldest, shift history	Impulse and constant rows
Vertical box sum	Reuses adjacent window rows	Per-column accumulated line state	Impulse at each row/window boundary
Candidate reducer	Selects minimum across lanes	Pipelined comparator tree	Permutations, all ties, last partial group
Best-state merge	Preserves global minimum across groups	Compare group winner with prior state	Better/worse/equal prior cost
Validity pipeline	Keeps metadata aligned	Register valid/metadata with each arithmetic stage	Random stalls and border warm-up

5.2 Compile-time parameters

Parameter	Meaning	Scaling effect
PIX_W	Input pixel width; initially 8	Subtractor and difference width
WIN_W, WIN_H	SAD window dimensions	Line/history storage and accumulator width
D_MAX	Maximum disparities searched	History depth, disparity width, number of groups
P_LANES	Parallel disparities per group	Logic, adders, taps, reducer size, pass count
FIFO_DEPTH	Transport elasticity	On-chip memory and stall tolerance
BURST_LEN	Requested SDRAM burst length	Efficiency versus buffering/controller limits

Derived widths must be local parameters, not magic constants:

$$DIFF_W = PIX_W$$

$$COST_MAX = (2^{PIX_W} - 1)WIN_WWIN_H$$

$$COST_W = \lceil \log_2(COST_MAX + 1) \rceil$$

$$DISP_W = \lceil \log_2(D_MAX) \rceil$$

For a first implementation, use zero-extension at adder boundaries and assertions that no derived width is undersized.

5.3 Scalability mechanism

For $P_LANES = P$, the number of raster disparity-group passes is approximately:

$$N_{groups} = \left\lceil \frac{D_{MAX}}{P} \right\rceil$$

Increasing P should reduce group passes but increases disparity taps, arithmetic lanes, reducer complexity, routing pressure, and potentially line/history memory ports. Therefore the resource-speedup curve—not ideal linear scaling—is the experimental result.

Level 4 — Proposed RTL module hierarchy

6. Module tree

LEVEL 4 — PROPOSED RTL MODULE TREE

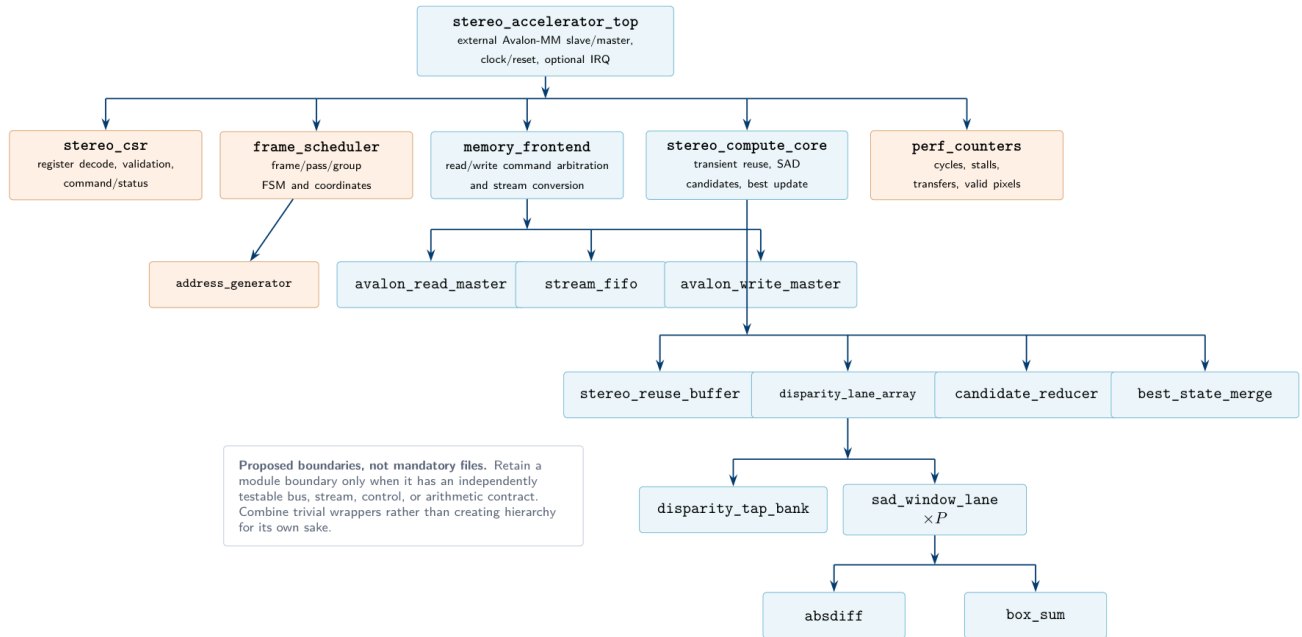


Figure 5 — Proposed independently testable implementation boundaries. These names are a design plan, not a requirement to create empty wrappers.

6.1 Top-level external ports

`stereo_accelerator_top` should expose only:

- Clock and synchronous/asynchronous reset as required by the platform.
- One Platform Designer-compatible control/status slave.
- One autonomous Avalon-MM memory master, or clearly arbitrated read/write master paths if the component packaging supports them.
- Optional completion/error IRQ.

Do not expose internal pixel/window signals at the Platform Designer boundary.

6.2 Proposed module contracts

Proposed module	Parent	Contract	First standalone test
stereo_csr	top	MM slave ↔ configuration/status record	Scratch, ID, start/busy/done
frame_scheduler	top	Command/status ↔ frame/pass control	Tiny frame and group sequencing
address_generator	scheduler	Logical request ↔ aligned bursts	Strides and boundary splits
memory_frontend	top	Commands/streams ↔ Avalon transactions	Pass-through buffer transform
avalon_read_master	frontend	Read command ↔ ordered response stream	Random wait/latency
avalon_write_master	frontend	Write stream ↔ accepted SDRAM writes	Random waitrequest
stream_fifo	frontend	valid/ready elasticity	Full/empty/simultaneous push-pop
stereo_reuse_buffer	compute core	Raster pixels ↔ aligned window/tap stream	Ramp/impulse borders
disparity_lane_array	compute core	Taps ↔ P candidate costs	P=1,2,4 generated instances
sad_window_lane	lane array	Pixel difference ↔ window SAD	Golden window sequence
candidate_reducer	compute core	P candidates ↔ group winner	Tie and permutation tests
best_state_merge	compute core	Prior + group winner ↔ updated best	Better/worse/equal cases
perf_counters	top	Event pulses ↔ readable snapshot	Simultaneous events and reset

If two proposed modules cannot be tested independently or have no meaningful protocol boundary, combine them. Hierarchy is used to isolate complexity, not to maximize file count.

Level 5 — Implementation and integration order

7. Vertical-slice build gates

Each gate should leave a demonstrable, recoverable system. Do not implement all leaf modules independently and attempt one final integration.

Gate	Integrated capability	Hardware contribution	Software/test contribution	Pass criterion
G0	Frozen contract	Parameter/format package	Integer golden model + vectors	All team members use identical rules
G1	Shared SDRAM platform	CPU + SDRAM controller	Destructive/non-destructive memory test	Address range and patterns pass
G2	Control plane	CSR slave, status, timer access	Driver register test	ID/scratch/start semantics pass
G3	Memory transport	Read/write master + FIFOs	Pass-through/copy benchmark	SDRAM block copied exactly under stalls
G4	One pixel-cost lane	Tap + <code>absdiff</code> , no full window	Directed row vectors	Per-pixel costs match golden model
G5	One complete SAD lane	Window accumulation + validity	Tiny frame comparison	Valid-region SAD costs match
G6	Complete P=1 disparity search	Reducer + best-state merge + group passes	Full disparity reference	Disparity map bit-exact
G7	Parameterized scaling	P=2, 4, ... generate + timing fixes	Automated benchmark table	Same output, measured tradeoff curve
G8	Continuous demonstration	Ping-pong/ring operation + counters	Batch launcher, report/VGA	Sustained frames, no manual reset

7.1 Why this order is safer

- G1–G3 retire the highest systems-integration risk before complex vision RTL exists.
- G4–G6 grow one mathematically verified datapath instead of debugging parallel lanes simultaneously.

- G7 changes performance, not functionality; all P values must preserve output equivalence.
- G8 is integration polish built on a working benchmark, not a substitute for correctness.

7.2 Team partition with explicit boundaries

Platform/integration owner

Owns Platform Designer, Nios V BSP, SDRAM, address map, CSR packaging, Avalon transport integration, and timing closure. Delivers tested bus/stream endpoints to the datapath owner.

Datapath/RTL owner

Owns reuse buffers, SAD lane, reducer, best-state merge, arithmetic widths, and RTL unit tests. Consumes and produces the agreed internal stream record only.

Software/evaluation owner

Owns golden model, vectors, dataset conversion, driver API, CPU baseline, cache handling, comparison, benchmark scripts, plots, and demonstration output.

Shared responsibilities

- All members freeze the arithmetic and memory contracts at G0.
 - Every owner supplies a self-checking test at their boundary.
 - Integration is performed at each gate, not postponed until the final week.
 - At least one non-owner reviews each block contract and result.
-

8. Clean project outline for proposal, report, and viva

Use this order when explaining the project:

1. **Problem and measurable research question.**
2. **Level-0 system contract and evaluation method.**
3. **Level-1 CPU/platform/accelerator/shared-memory partition.**
4. **Why stereo SAD is suitable for streaming hardware.**

5. **Level-2 accelerator data, control, and status paths.**
6. **Level-3 parameterized SAD datapath and arithmetic.**
7. **Level-4 proposed RTL module contracts.**
8. **Shared-SDRAM map, coherency, and timing boundary.**
9. **Verification ladder from unit vectors to FPGA datasets.**
10. **Vertical-slice build gates and team responsibilities.**
11. **Scalability experiment and resource/performance curve.**
12. **Demo flow, risks, results, and conclusions.**

This order prevents three common problems:

- Explaining low-level adders before the audience knows the system objective.
 - Mixing software tasks with RTL modules in one unstructured list.
 - Claiming scalability without connecting `P_LANES` to both architecture and measurement.
-

9. Immediate next action

The next implementation action remains **G1: prove the intended external SDRAM region from Nios V**. Only after that passes should the team package the G2 control slave and G3 pass-through memory master. The SAD datapath can be developed in simulation in parallel, but it should not be connected to an unverified memory system.